



UNIVERSIDAD SERGIO ARBOLEDA

**DESEMPEÑO Y COMPARACIÓN PSO VS. ACO
PARA EL PROBLEMA DEL VIAJANTE (TSP)**

Inteligencia Artificial

Camilo Andrés Díaz García

*Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Introducción	2
2. Metodología de PSO	2
3. Resultados por instancia	2
4. Comparación PSO vs. ACO	3
5. Conclusiones	3

1 Introducción

Este trabajo implementa PSO (*Particle Swarm Optimization*) para el TSP mediante codificación de claves aleatorias, en las mismas tres instancias del proyecto ACO-IA ($n=20$, 2 000 y 200 000 ciudades), midiendo tiempo y memoria, y comparando los resultados directamente contra las cifras ya obtenidas con ACO en ese proyecto.

2 Metodología de PSO

Cada partícula tiene un vector de posición continuo $\mathbf{x}_i \in \mathbb{R}^n$ (una clave por ciudad). Para evaluarla, se ordenan las ciudades por el valor de su clave (*argsort*); ese orden es el tour, evaluado con la misma función objetivo de ACO ($L(\text{tour}) = \sum d(c_i, c_{i+1}) + d(c_n, c_1)$, distancia euclidiana). La actualización es la estándar:

$$\mathbf{v}_i(t+1) = w \mathbf{v}_i(t) + c_1 r_1 [\mathbf{p}_i - \mathbf{x}_i(t)] + c_2 r_2 [\mathbf{g} - \mathbf{x}_i(t)], \quad (1)$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1), \quad (2)$$

con w (inercia), c_1 (cognitivo), c_2 (social) y $r_1, r_2 \sim U(0, 1)$ independientes por componente. $\mathbf{p}_i$ es la mejor posición personal de la partícula i y $\mathbf{g}$ la mejor global del enjambre. Las claves se recortan a $[-50, 50]$ (con reinicio de velocidad a 0 al recortar) si una componente se dispara; no hay más tratamiento de límites porque las claves solo importan por su orden relativo.

Parámetros usados en las tres instancias: $w = 0.7$, $c_1 = c_2 = 1.5$, inicialización $\mathbf{x}_i \sim U(0, 1)^n$, $\mathbf{v}_i(0) = \mathbf{0}$. El número de partículas e iteraciones se ajustó por instancia (partículas moderadas, más iteraciones, no un enjambre gigantesco) y se documenta en la Tabla 1. Implementado en C++20 con OpenMP, paralelizando la evaluación de partículas por hilo con un buffer reutilizado (sin reasignar memoria en el bucle caliente), igual que en ACO.

3 Resultados por instancia

Tabla 1: Resultados de PSO por instancia (semillas 1–3 para $n=2\,000$ y $n=200\,000$; 1–5 para $n=20$).

Instancia	Partículas	Iteraciones	L_{mejor} media	Tiempo medio (s)	Memoria pico (MB)
$n = 20$	200	300	4,344 (desv. 0,576)	0,131	4,5
$n = 2\,000$	200	500	978,627 (desv. 19,578)	2,264	9,4
$n = 200\,000$	100	100	103 879,712 (desv. 73,322)	41,606	266,3

En $n = 20$, PSO se validó contra el óptimo exacto de Held-Karp (calculado en ACO-IA): con 200 partículas y 300 iteraciones, 3 de 5 semillas llegan a un gap menor al 1 %, pero 2 de

5 quedan entre 11 % y 23 % — la calidad depende fuertemente de la semilla, a diferencia de ACO. En $n = 2000$ y $n = 200000$ no hay óptimo exacto disponible; la comparación relevante es contra ACO (sección 4). En las tres instancias todos los tours se verificaron como permutaciones válidas.

4 Comparación PSO vs. ACO

Las cifras de ACO son las oficiales del proyecto ACO_IA (commit 5613b16): $n=20$ con $m=20$, $K=19$ (5 semillas, contra Held-Karp); $n=2000$ con $m=2000$, $K=8$ (calidad: 8 semillas; tiempo y memoria: 3 semillas, misma configuración); $n=200000$ con $m=20000$ y con $m=2048$, ambas ya documentadas en ese informe.

Tabla 2: PSO vs. ACO por instancia.

Instancia	Algoritmo	L_{mejor} media	Tiempo medio (s)	Memoria (MB)
$n = 20$	ACO ($m=20$, $K=19$)	4,060	(nota 1)	(nota 1)
$n = 20$	PSO (200 part., 300 it.)	4,344	0,131	4,5
$n = 2000$	ACO ($m=2000$)	37,496	4,451	5,6
$n = 2000$	PSO (200 part., 500 it.)	978,627	2,264	9,4
$n = 200000$	ACO ($m=20000$, 10 it.)	387,308	838,3	102,4
$n = 200000$	ACO ($m=2048$, 108 it.)	385,485	904,8	102,4
$n = 200000$	PSO (100 part., 100 it.)	103 879,712	41,6	266,3

Nota 1: ACO no registró tiempo/memoria por separado para la configuración exacta de $n = 20$ ($m=20$, $K=19$) en el proyecto original; la referencia más cercana disponible en ese proyecto para $n = 20$ (misma máquina, $m=2048$, $K=10$) fue 0,021 s y 4,5 MB.

En $n = 20$, ambos quedan cerca del óptimo (ACO ligeramente mejor y más consistente entre semillas); el tiempo no es comparable de forma directa porque ACO no midió esa configuración exacta por separado. En $n = 2000$ y sobre todo en $n = 200000$, ACO da una calidad muchísimo mejor (en $n = 200000$, el tour de ACO es del orden de 270 veces más corto que el de PSO), mientras que PSO es más rápido ($2\times$ en $n = 2000$, $\approx 20\times$ en $n = 200000$) y usa más memoria ($\approx 1.7\times$ y $\approx 2.6\times$ respectivamente). La codificación de claves aleatorias no tiene la ventaja estructural de la feromona sobre aristas del grafo, y esa desventaja crece con n .

5 Conclusiones

1. PSO con claves aleatorias converge razonablemente en instancias pequeñas ($n = 20$), pero su calidad se degrada drásticamente al escalar: en $n = 200000$ el tour es dos órdenes de magnitud más largo que el de ACO.

2. PSO es computacionalmente más barato por corrida (más rápido, aunque con más memoria pico) precisamente porque no mantiene ni actualiza información por arista como la feromona de ACO — y esa misma ausencia es la causa de su peor calidad a gran escala.
3. Para TSP a esta escala, ACO (con lista de K candidatas y feromona por arista) es la opción claramente superior; PSO con codificación de claves aleatorias no está bien adaptado a un problema combinatorio de esta naturaleza sin modificaciones adicionales (por ejemplo, búsqueda local o un operador de cruce discreto), fuera del alcance de este trabajo.